Python Programming

NumPy & Linear Algebra

Hoa T. Vu

Variables

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10           # int
pi = 3.14        # float
name = "Alice"   # str
flag = True      # bool
```

```
print(x)         # 10
print(type(x))   # <class 'int'>

x = x + 1       # reassign
x += 1           # shorthand
```

Variables

A variable stores a value. Python infers the type automatically.

```
x = 10          # int
pi = 3.14       # float
name = "Alice"  # str
flag = True     # bool
```

```
print(x)          # 10
print(type(x))    # <class 'int'>

x = x + 1       # reassign
x += 1           # shorthand
```

- No declaration needed, just assign
- Types can change: `x = 10` then `x = "hello"` is valid

Common Operations

Arithmetic

```
7 + 2    # 9
7 - 2    # 5
7 * 2    # 14
7 / 2    # 3.5   (float)
7 // 2   # 3     (floor div)
7 % 2    # 1     (remainder)
7 ** 2   # 49    (power)
```

Strings

```
s = "hello"
len(s)           # 5
s.upper()        # "HELLO"
s[0]             # "h"
s[1:4]           # "ell"

# f-string
n = 42
f"answer: {n}"   # "answer: 42"
```

Lists

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]

nums[0]           # 3         (first element)
nums[-1]          # 5         (last element)
nums[1:3]         # [1, 4]    (slice)
len(nums)         # 5

nums.append(9)    # [3, 1, 4, 1, 5, 9]
nums.pop()        # removes and returns 9
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists

An ordered, mutable collection of items.

```
nums = [3, 1, 4, 1, 5]

nums[0]           # 3         (first element)
nums[-1]          # 5         (last element)
nums[1:3]         # [1, 4]    (slice)
len(nums)         # 5

nums.append(9)    # [3, 1, 4, 1, 5, 9]
nums.pop()        # removes and returns 9
nums.sort()       # sorts in place: [1, 1, 3, 4, 5]
```

Lists can hold mixed types: `[1, "hello", 3.14, True]`

List Comprehension

Build a list in one line more readable than a loop.

```
# squares of 0..4
squares = [x**2 for x in range(5)]
# [0, 1, 4, 9, 16]

# even numbers only
evens = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]

# nested: 2-D grid of (row, col) pairs
pairs = [(r, c) for r in range(3) for c in range(3)]
```

Conditionals

if / elif / else

Python uses **indentation** (4 spaces) to define blocks, no braces.

```
score = 78

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "F"

print(grade)  # "C"
```

Comparison operators

- `==` equal
- `!=` not equal
- `<`, `<=`, `>`, `>=`
- `and`, `or`, `not`
- `in` membership

Loops

for and while

for loop

```
# iterate over a list
for x in [2, 4, 6]:
    print(x)

# iterate with index
for i, x in enumerate([2,4,6]):
    print(i, x)

# range
for i in range(5):     # 0..4
    print(i)
```

while loop

```
n = 1
while n < 100:
    n *= 2
# n = 128

# break / continue
for x in range(10):
    if x == 5:
        break       # exit loop
    if x % 2 == 0:
        continue    # skip even
    print(x)   # 1 3
```

NumPy

NumPy Arrays

NumPy provides fast, vectorized operations on arrays.

```
import numpy as np

a = np.array([1, 2, 3, 4])    # 1-D array
A = np.array([[1, 2],
              [3, 4]])         # 2-D array (matrix)

a.shape     # (4,)
A.shape     # (2, 2)
A.dtype     # float64 / int64 ...

np.zeros((3, 3))    # 3x3 zero matrix
np.ones((2, 4))     # 2x4 ones
np.eye(3)           # 3x3 identity
np.random.randn(3, 3)  # random normal
```

Indexing and Slicing

```
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

A[0, 1]       # 2          (row 0, col 1)
A[1, :]       # [4, 5, 6]  (entire row 1)
A[:, 2]       # [3, 6, 9]  (entire col 2)
A[:2, :2]     # [[1,2],[4,5]]  (top-left 2x2)

# boolean mask
a = np.array([1, -2, 3, -4])
a[a > 0]      # [1, 3]
a[a > 0] = 0 # set positives to zero
```

Vectorized Operations

Operations apply **element-wise** — no explicit loops needed.

```
a = np.array([1.0, 2.0, 3.0])
b = np.array([4.0, 5.0, 6.0])

a + b          # [5., 7., 9.]
a * b          # [4., 10., 18.]
a ** 2         # [1., 4., 9.]
np.sqrt(a)     # [1., 1.41, 1.73]
np.exp(a)      # [e, e^2, e^3]

# scalar broadcast
a * 3          # [3., 6., 9.]
a + 10         # [11., 12., 13.]
```

Linear Algebra

```
A = np.array([[1., 2.], [3., 4.]])
b = np.array([5., 6.])

# dot product / matrix-vector product
np.dot(a, b)       # scalar
A @ b              # matrix-vector: shape (2,)
A @ A              # matrix-matrix product

# transpose
A.T                # [[1,3],[2,4]]

# norms
np.linalg.norm(b)          # L2 norm
np.linalg.norm(b, ord=1)   # L1 norm

# solve Ax = b
x = np.linalg.solve(A, b)
```

Reductions and Statistics

```
A = np.array([[1., 2., 3.],
              [4., 5., 6.]])

A.sum()           # 21.0  (all elements)
A.sum(axis=0)     # [5., 7., 9.]  (column sums)
A.sum(axis=1)     # [6., 15.]     (row sums)

A.mean()          # 3.5
A.max()           # 6.0
A.argmax()        # 5   (flat index)
A.argmax(axis=1)  # [2, 2]  (col index of max per row)
```

Reductions and Statistics

```
A = np.array([[1., 2., 3.],
              [4., 5., 6.]])

A.sum()           # 21.0  (all elements)
A.sum(axis=0)     # [5., 7., 9.]  (column sums)
A.sum(axis=1)     # [6., 15.]     (row sums)

A.mean()          # 3.5
A.max()           # 6.0
A.argmax()        # 5   (flat index)
A.argmax(axis=1)  # [2, 2]  (col index of max per row)
```

Rule of thumb: if you are writing a Python `for` loop over array elements, there is usually a NumPy function that does it faster.

Problems

Problem 1 — Lists

Given `scores = [88, 72, 95, 61, 84, 90]`, write code to:

1. Access the third score.
2. Slice the last three scores.
3. Append 77, then sort in descending order.
4. Remove all scores below 80 using a list comprehension.

Problem 2 — Loops

1. Write a loop that prints the first 10 Fibonacci numbers:
   1, 1, 2, 3, 5, 8, 13, 21, 34, 55
2. Modify it to find the **first** Fibonacci number greater than 1 000.
3. Write a loop that finds all prime numbers up to 50.
   Hint: a number n is prime if no integer from 2 to $\sqrt{n}$ divides it.

Problem 3 — NumPy Linear Algebra

```
import numpy as np
A = np.array([[2., 1.], [5., 3.]])
b = np.array([4., 7.])
```

1. Solve $Ax = b$ with `np.linalg.solve`. Verify by checking $Ax \approx b$.
2. Compute the L1 and L2 norms of x.
3. Compute the eigenvalues of A.
4. What is `np.linalg.cond(A)`? What does a large condition number mean for numerical solving?